

Prática introdutória 01

```
#include <math.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define GRUPO_DERIVADA 0
#define GRUPO_INTEGRAL 1

/* funções auxiliares */
double f(double x) {
    return x * x + 3.0 * x + 2.0;
}

double calcular_derivada_diferencas_finitas(double x, double h) {
    return (f(x + h) - f(x - h)) / (2.0 * h);
}

/* função principal para cálculo da derivada */
void executar_calculo_derivada(MPI_Comm subcomunicador) {
    int sub_rank, sub_size;

    // seleciona os novos ranks do subcomunicador
    MPI_Comm_rank(subcomunicador, &sub_rank);
    MPI_Comm_size(subcomunicador, &sub_size);

    const int total_pontos = 10;
    const double inicio = 0.0;
    const double fim = 9.0;
    const double h = 1e-5;

    double soma_local = 0.0;
    int quantidade_local = 0;

    // for para calcular a derivada aproximada em cada ponto, distribuindo
    // os pontos entre os processos do subcomunicador
    for (int i = sub_rank; i < total_pontos; i += sub_size) {
        double x = inicio + i * ((fim - inicio) / (total_pontos - 1));
        double derivada = calcular_derivada_diferencas_finitas(x, h);

        printf("[DERIVADA] Rank local %d calculou f'(%0.2f) = %0.6f\n",
            sub_rank, x, derivada);

        soma_local += derivada;
        quantidade_local++;
    }

    double soma_global = 0.0;
    int quantidade_global = 0;
```

```
// reduz os resultados locais para o processo 0 do subcomunicador
usando MPI_Reduce
MPI_Reduce(&soma_local, &soma_global, 1, MPI_DOUBLE, MPI_SUM, 0,
subcomunicador);
MPI_Reduce(&quantidade_local, &quantidade_global, 1, MPI_INT, MPI_SUM,
0, subcomunicador);

// o processo 0 do subcomunicador imprime a soma e a média das
derivadas aproximadas
if (sub_rank == 0) {
    printf("\n[DERIVADA] Soma das derivadas aproximadas = %.6f\n",
soma_global);
    printf("[DERIVADA] Media das derivadas aproximadas = %.6f\n\n",
        soma_global / quantidade_global);
}
}

// função principal para cálculo da integral
void executar_calculo_integral(MPI_Comm subcomunicador) {

    // seleciona os novos ranks do subcomunicador
    int sub_rank, sub_size;
    MPI_Comm_rank(subcomunicador, &sub_rank);
    MPI_Comm_size(subcomunicador, &sub_size);

    const double a = 0.0;
    const double b = 10.0;
    const int n = 100000;
    const double largura = (b - a) / n;

    double area_local = 0.0;

    // for para calcular a área aproximada usando o método dos trapézios,
    distribuindo os pontos entre os processos do subcomunicador
    for (int i = sub_rank; i < n; i += sub_size) {
        double x_i = a + i * largura;
        double x_proximo = x_i + largura;

        area_local += ((f(x_i) + f(x_proximo)) / 2.0) * largura;
    }

    // cada processo do subcomunicador tem sua área local calculada
    // uso do MPI_Allgather para coletar todas as áreas locais em um vetor
    no processo 0 do subcomunicador
    double *areas_parciais = malloc((size_t)sub_size * sizeof(double));
    if (areas_parciais == NULL) {
        fprintf(stderr, "[INTEGRAL] Erro ao alocar vetor de areas
parciais.\n");
        MPI_Abort(subcomunicador, EXIT_FAILURE);
    }

    // coleta as áreas locais de todos os processos do subcomunicador com o
    MPI_Allgather
```

```
MPI_Allgather(&area_local, 1, MPI_DOUBLE,
              areas_parciais, 1, MPI_DOUBLE,
              subcomunicador);

double area_total = 0.0;

// soma as áreas parciais para obter a área total aproximada da
integral
for (int i = 0; i < sub_size; i++) {
    area_total += areas_parciais[i];
}

printf("[INTEGRAL] Rank local %d calculou area parcial = %.6f\n",
       sub_rank, area_local);

// o processo 0 do subcomunicador imprime as áreas parciais e a área
total aproximada da integral
if (sub_rank == 0) {
    printf("\n[INTEGRAL] Areas recebidas com MPI_Allgather:");
    for (int i = 0; i < sub_size; i++) {
        printf(" %.6f", areas_parciais[i]);
    }
    printf("\n[INTEGRAL] Integral aproximada de f(x) em [%.2f, %.2f] =
%.6f\n\n",
          a, b, area_total);
}

free(areas_parciais);
}

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank_global, size_global;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank_global);
    MPI_Comm_size(MPI_COMM_WORLD, &size_global);

    if (size_global < 2) {
        if (rank_global == 0) {
            printf("Erro: execute com pelo menos 2 processos.\n");
            printf("Exemplo: mpirun -np 4 ./mpi_reduce_allgather\n");
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }

    int metade = size_global / 2;
    int cor_grupo = (rank_global < metade) ? GRUPO_DERIVADA :
GRUPO_INTEGRAL;

    // cria um subcomunicador para cada grupo usando MPI_Comm_split
    MPI_Comm subcomunicador;
    MPI_Comm_split(MPI_COMM_WORLD, cor_grupo, rank_global,
&subcomunicador);
```

```
// o processo 0 do comunicador global imprime informações sobre os
grupos
if (rank_global == 0) {
    printf("Programa MPI com subgrupos\n");
    printf("Total de processos: %d\n", size_global);
    printf("Processos no grupo da derivada: %d\n", metade);
    printf("Processos no grupo da integral: %d\n\n", size_global -
metade);
}

// sincroniza os processos antes de iniciar os cálculos
MPI_Barrier(MPI_COMM_WORLD);

// cada grupo executa sua função específica (derivada ou integral)
usando o subcomunicador criado
if (cor_grupo == GRUPO_DERIVADA) {
    executar_calculo_derivada(subcomunicador);
} else {
    executar_calculo_integral(subcomunicador);
}

MPI_Comm_free(&subcomunicador);
MPI_Finalize();

return EXIT_SUCCESS;
}
```